

VOLUME 1

Kubernetes · OpenShift · JFrog · ArgoCD

Architecture, commandes essentielles, AKS, ARO, gestion des artefacts et GitOps sur Azure



Kubernetes
Fondamentaux



AKS Azure K8s
Service



OpenShift ARO sur
Azure



JFrog Artifactory



CI/CD Pipelines



ArgoCD GitOps

① KUBERNETES — Architecture & Composants Fondamentaux

Présentation des 7 composants internes du cluster Kubernetes (plan de contrôle et nœuds workers), leur rôle précis et les commandes pour vérifier leur état. Indispensable pour comprendre ce qui se passe « sous le capot » avant de travailler sur AKS ou OpenShift.

Composant	Rôle	Commande de vérification
API Server	Point d'entrée unique du cluster. Valide et traite toutes les requêtes REST. Authentification, autorisation (RBAC), admission controllers.	<code>kubectl cluster-info</code> <code>kubectl version --short</code> <code>kubectl api-resources</code>
etcd	Base de données clé-valeur distribuée (Raft). Stocke l'état complet du cluster. Critique : sauvegarder régulièrement.	<code>kubectl get componentstatus # Backup etcd (accès direct node master)</code> <code>etcdctl snapshot save backup.db</code>
Controller Manager	Boucles de réconciliation : ReplicaSet, Deployment, Node, Job, CronJob... Compare état désiré vs état réel.	<code>kubectl get pods -n kube-system Select-String 'controller-manager'</code>
Scheduler	Affecte les Pods sur les Nodes selon : ressources disponibles, affinités, taints/tolerations, priorité.	<code>kubectl describe pod <pod> Select-String 'Node:'</code> <code>kubectl get events --field-selector reason=Scheduled</code>
Kubelet	Agent sur chaque Node. Communique avec l'API Server. Gère le cycle de vie des conteneurs via CRI (containerd/CRI-O).	<code>kubectl get nodes -o wide</code> <code>kubectl describe node <node-name></code>
kube-proxy	Gère les règles réseau (iptables/ipvs) pour router le trafic vers les Services. Un pod par nœud (DaemonSet).	<code>kubectl get ds kube-proxy -n kube-system</code> <code>kubectl get svc -A</code>
CoreDNS	DNS interne du cluster. Résout ..svc.cluster.local. Configurable via ConfigMap.	<code>kubectl get cm coredns -n kube-system -o yaml</code> <code>kubectl exec -it <pod> -- nslookup kubernetes</code>

Objets Kubernetes — Référence Rapide

Catalogue des 6 objets K8s les plus utilisés au quotidien (Pod, Deployment, Service, ConfigMap, Secret, HPA) avec pour chacun : un exemple de YAML minimal prêt à l'emploi et les commandes kubectl essentielles.

Objet	Usage	YAML Minimal	Commandes clés
Pod	Unité de base — 1+ conteneurs partageant réseau/stockage	<code>apiVersion: v1</code> <code>kind: Pod</code> <code>metadata: name: mon-pod</code> <code>spec: containers: - name: app image: nginx:latest ports: - containerPort: 80</code>	<code>kubectl run mon-pod --image=nginx</code> <code>kubectl get pod mon-pod -o wide</code> <code>kubectl exec -it mon-pod -- bash</code> <code>kubectl delete pod mon-pod</code>

Objet	Usage	YAML Minimal	Commandes clés
Deployment	Gère ReplicaSets pour rolling updates et rollbacks	apiVersion: apps/v1 kind: Deployment metadata: name: mon-app spec: replicas: 3 selector: matchLabels: app: mon-app template: metadata: labels: app: mon-app spec: containers: - name: app image: mon-app:v1	kubectl apply -f deploy.yaml kubectl rollout status deploy/mon-app kubectl rollout undo deploy/mon-app kubectl scale deploy/mon-app --replicas=5
Service	Exposition stable (ClusterIP/ NodePort/LoadBalancer/ExternalName)	apiVersion: v1 kind: Service metadata: name: mon-svc spec: selector: app: mon-app ports: - port: 80 targetPort: 8080 type: ClusterIP	kubectl expose deploy mon-app --port=80 kubectl get svc -o wide kubectl port-forward svc/mon-svc 8080:80
ConfigMap	Configuration non-sensible injectée en variables env ou volume	apiVersion: v1 kind: ConfigMap metadata: name: app-config data: ENV: production DB_HOST: postgres-svc app.properties: log.level=INFO	kubectl create cm app-config \ --from-literal=ENV=prod \ --from-file=app.properties kubectl get cm app-config -o yaml
Secret	Données sensibles encodées base64 (mot de passe, token, cert)	apiVersion: v1 kind: Secret metadata: name: db-secret type: Opaque data: password: UzNjcjN0IQ== # base64	kubectl create secret generic db-secret \ --from-literal=password='S3cr3t!' kubectl get secret db-secret -o jsonpath='{.data.password}' base64 -d
HPA	Horizontal Pod Autoscaler — scale selon CPU/mémoire/métriques custom	apiVersion: autoscaling/v2 kind: HorizontalPodAutoscaler metadata: name: mon-hpa spec: scaleTargetRef: apiVersion: apps/v1 kind: Deployment name: mon-app minReplicas: 2 maxReplicas: 10 metrics: - type: Resource resource: name: cpu target: type: Utilization averageUtilization: 70	kubectl autoscale deploy mon-app \ --min=2 --max=10 --cpu-percent=70 kubectl get hpa kubectl describe hpa mon-hpa

② AKS — Azure Kubernetes Service : Setup & Opérations

Toutes les étapes pour provisionner et opérer un cluster AKS sur Azure : connexion, création avec identité managée et CNI Azure, gestion des node pools, attachement d'un registry ACR, upgrade de version, activation de KEDA pour l'autoscaling événementiel, et mise en place du monitoring via Container Insights.

Opération	Script PowerShell / Azure CLI	Notes importantes
Connexion Azure	<pre># PowerShell Connect-AzAccount Set-AzContext -SubscriptionId '<sub-id>' # Azure CLI az login az account set --subscription '<sub-id>' az account show</pre>	Utiliser un Service Principal en CI/CD: az ad sp create-for-rbac --role Contributor
Créer cluster AKS	<pre>az aks create \ --resource-group rg-prod \ --name aks-prod \ --node-count 3 \ --node-vm-size Standard_D4s_v3 \ --enable-addons monitoring \ --enable-managed-identity \ --network-plugin azure \ --network-policy azure \ --enable-auto-upgrade \ --generate-ssh-keys</pre>	• Managed Identity : évite de gérer les credentials • azure CNI : meilleure intégration VNet • network-policy azure : isolation pods
Obtenir kubeconfig	<pre>az aks get-credentials \ --resource-group rg-prod \ --name aks-prod \ --overwrite-existing # Vérification kubectl get nodes kubectl cluster-info</pre>	Fusionne dans ~/.kube/config Utiliser --admin pour accès admin complet
Node Pools	<pre># Ajouter pool spécialisé az aks nodepool add \ --resource-group rg-prod \ --cluster-name aks-prod \ --name workerpool \ --node-count 2 \ --node-vm-size Standard_D8s_v3 \ --mode User # Scaler un pool az aks nodepool scale \ --node-count 5 \ --name workerpool \ --cluster-name aks-prod \ --resource-group rg-prod</pre>	Mode System : pods système (CoreDNS, etc.) Mode User : workloads applicatifs Nodepool dédié GPU, spot, etc.
Attacher ACR	<pre># Attacher Azure Container Registry az aks update \ --name aks-prod \ --resource-group rg-prod \ --attach-acr acrprodregistry # Vérifier az aks show -g rg-prod -n aks-prod \ --query identityProfile</pre>	Évite de créer des imagePullSecrets Utilise l'identité managée du cluster
Upgrade cluster	<pre># Voir versions disponibles az aks get-upgrades \ --resource-group rg-prod \ --name aks-prod --output table # Upgrade az aks upgrade \ --resource-group rg-prod \ --name aks-prod \ --kubernetes-version 1.29.0</pre>	Upgrade control plane puis node pools Planifier en fenêtre de maintenance Tester d'abord en environnement dev

Opération	Script PowerShell / Azure CLI	Notes importantes
Monitoring / Logs	<pre># Activer Container Insights az aks enable-addons \ -a monitoring \ -g rg-prod -n aks-prod # Requête Log Analytics (PowerShell) \$wsId = (az aks show -g rg-prod -n aks-prod \ --query addonProfiles.omsagent.config.logAnalyticsWorkspaceResourceID \ -o tsv) \$query = 'KubePodInventory where TimeGenerated > ago(1h) summarize count() by Namespace' Invoke-AzOperationalInsightsQuery -WorkspaceId \$wsId -Query \$query</pre>	Container Insights → Log Analytics Tableau de bord Azure Monitor Alertes sur CPU/mémoire/redémarrages
KEDA (Event-driven scale)	<pre># Activer KEDA sur AKS az aks update \ --resource-group rg-prod \ --name aks-prod \ --enable-keda # ScaledObject exemple (Azure Service Bus) apiVersion: keda.sh/v1alpha1 kind: ScaledObject metadata: name: sb-scaler spec: scaleTargetRef: name: mon-app triggers: - type: azure-servicebus metadata: queueName: ma-queue messageCount: '5'</pre>	Scale à 0 quand pas de messages Supporte: Service Bus, Event Hub, Storage Queue, HTTP, Prometheus...

③ OpenShift (ARO) sur Azure — Installation & Commandes

Guide complet pour déployer et utiliser Azure Red Hat OpenShift (ARO) : installation du cluster via Azure CLI, login avec la CLI oc, gestion des projets (namespaces enrichis), déploiement d'applications avec S2I, configuration des Routes HAProxy (équivalent Ingress), Security Context Constraints (SCC), OperatorHub et ImageStreams — avec mise en évidence des différences clés par rapport à Kubernetes vanilla.

Opération	Commande oc / PowerShell / YAML	Spécificité OpenShift vs K8s
Installer ARO	<pre># Prérequis : VNet + subnets dédiés az network vnet create -g rg-aro -n aro-vnet --address-prefixes 10.0.0.0/8 az network vnet subnet create -g rg-aro --vnet-name aro-vnet -n master-subnet --address-prefixes 10.0.0.0/23 az network vnet subnet create -g rg-aro --vnet-name aro-vnet -n worker-subnet --address-prefixes 10.0.2.0/23 # Créer le cluster ARO az aro create \ --resource-group rg-aro \ --name cluster-aro \ --vnet aro-vnet \ --master-subnet master-subnet \ --worker-subnet worker-subnet \ --pull-secret @pull-secret.txt</pre>	ARO = OpenShift managé par Microsoft + Red Hat Nécessite Red Hat pull-secret (account.redhat.com) Masters non accessibles (fully managed)
Login & Credentials	<pre># Récupérer URL console az aro show -g rg-aro -n cluster-aro \ --query consoleProfile.url -o tsv # Récupérer credentials admin \$creds = az aro list-credentials \ -g rg-aro -n cluster-aro ConvertFrom-Json # Login CLI oc login \$url \ -u \$creds.kubeAdminUsername \ -p \$creds.kubeAdminPassword # Vérification oc whoami oc get clusterversion</pre>	Console web accessible via URL kubeadmin = compte admin temporaire Créer un Identity Provider (AAD) en production
Projets (Namespaces++)	<pre>oc new-project mon-projet \ --description='App production' \ --display-name='Mon Application' oc project mon-projet oc get projects oc describe project mon-projet # Quota automatique sur projet oc describe quota -n mon-projet oc describe limitrange -n mon-projet</pre>	Projects = Namespaces + métadonnées Quotas et LimitRanges auto-appliqués Pas besoin de créer le namespace manuellement
Déployer application	<pre># Depuis image Docker oc new-app --docker-image=acrprod.azurecr.io/app:v1 # Depuis code source (S2I) oc new-app https://github.com/org/repo.git --name=mon-app # Depuis Dockerfile local oc new-build --binary --name=mon-app oc start-build mon-app --from-dir=. --follow # Suivre le statut oc status oc get all -n mon-projet</pre>	S2I (Source-to-Image) = build intégré new-app crée : Deployment + Service + ImageStream Pas besoin d'un registry externe pour les builds
Routes (Ingress OCP)	<pre># Exposer un service oc expose svc/mon-app # Route HTTPS (edge TLS) oc create route edge mon-app-https \ --service=mon-app \ --cert=tls.crt \ --key=tls.key \ --hostname=mon-app.apps.aro.example.com # Vérifier oc get routes oc describe route mon-app-https # Annoter pour load balancing oc annotate route mon-app \ haproxy.router.openshift.io/balance=roundrobin</pre>	3 modes TLS : • edge : TLS terminé sur router HAProxy • passthrough : TLS jusqu'au pod • reencrypt : re-chiffrement interne Wildcard DNS auto : *.apps.

Opération	Commande oc / PowerShell / YAML	Spécificité OpenShift vs K8s
Security Context Constraints	# Lister les SCCs oc get scc # Appliquer SCC anyuid à un SA oc adm policy add-scc-to-serviceaccount \ anyuid -z mon-sa -n mon-projet # Voir les SCCs d'un pod oc get pod <pod> -o yaml grep -i scc # Déboguer SCC oc adm policy who-can use scc anyuid	OCP applique restricted-v2 par défaut (plus strict que K8s vanilla) Containeurs doivent tourner en non-root RunAsUser: 1000650000+ (range alloué)
OperatorHub & Operators	# Lister les operators installés oc get csv -n openshift-operators # Chercher un operator (ex: PostgreSQL) oc get packagemanifest -n openshift-marketplace \ Select-String 'postgres' # Installer via subscription YAML apiVersion: operators.coreos.com/v1alpha1 kind: Subscription metadata: name: postgresql-operator namespace: openshift-operators spec: channel: stable name: postgresql source: redhat-operators sourceNamespace: openshift-marketplace	OLM (Operator Lifecycle Manager) intégré Catalogues : Red Hat, Certified, Community Gestion cycle de vie (install/upgrade/delete)
ImageStreams	# Importer depuis ACR oc import-image mon-app:latest \ --from=acrprod.azurecr.io/app:latest \ --confirm # Tag une image oc tag mon-app:latest mon-app:stable # Déclencher build sur push oc set triggers deploy/mon-app \ --from-image=mon-app:latest # Voir les images oc get is oc describe is mon-app	ImageStream = abstraction au-dessus du registry Déclencheurs automatiques de déploiement Versatile : local, ACR, Docker Hub, Quay...

④ JFrog Artifactory sur Azure — Gestion des Artefacts & Registry

Installation de JFrog Artifactory sur AKS via Helm, configuration de l'API avec tokens d'accès, création de dépôts locaux/virtuels/remote pour Docker, Helm et autres formats. Couvre aussi le push/pull d'images depuis Kubernetes, l'analyse de sécurité CVE avec Xray, et les politiques de rétention pour nettoyer les anciens artefacts.

Opération	Commande / Script PowerShell	Description & Bonnes Pratiques
Installer JFrog sur AKS	<pre>helm repo add jfrog https://charts.jfrog.io helm repo update helm install artifactory jfrog/artifactory \ --namespace jfrog --create-namespace \ --set artifactory.persistence.storageClassName=managed-premium \ --set artifactory.persistence.size=200Gi \ --set artifactory.replicaCount=2 \ --set nginx.enabled=true \ --set nginx.service.type=LoadBalancer \ -f values-custom.yaml</pre>	Storage : Azure Disk (managed-premium) ou Azure Blob HA : minimum 2 replicas + base PostgreSQL externe Ingress : configurer avec cert-manager pour HTTPS
Configuration API	<pre># Variables d'environnement \$env:JFROG_URL = 'https://artifactory.mon-domaine.com' \$env:JFROG_TOKEN = '<access-token>' # Headers réutilisables \$headers = @{ 'Authorization' = "Bearer \$env:JFROG_TOKEN" 'Content-Type' = 'application/json' } # Test de connexion Invoke-RestMethod \ -Uri "\$env:JFROG_URL/api/system/ping" \ -Headers \$headers</pre>	Préférer Access Token à l'API Key (déprécié) Token : Admin → Identity and Access → Access Tokens Scopé par permissions et expiration
Créer dépôts	<pre># Dépôt local Docker \$repoConfig = @{ rclass = 'local' packageType = 'docker' repoLayoutRef = 'simple-default' description = 'Images Docker production' } ConvertTo-Json Invoke-RestMethod \ -Uri "\$env:JFROG_URL/api/repositories/docker-local-prod" \ -Method PUT \ -Headers \$headers \ -Body \$repoConfig # Dépôt virtuel (agrège local + remote) \$repoConfig = @{ rclass = 'virtual' packageType = 'docker' repositories = @('docker-local-prod', 'docker-remote-hub') defaultDeploymentRepo = 'docker-local-prod' } ConvertTo-Json Invoke-RestMethod \ -Uri "\$env:JFROG_URL/api/repositories/docker-virtual" \ -Method PUT \ -Headers \$headers \ -Body \$repoConfig</pre>	3 types de dépôts : • Local : stockage interne JFrog • Remote : proxy vers Docker Hub, MCR... • Virtual : façade unifiée local+remote Packages : docker, helm, npm, maven, pypi, nuget, generic
Push/Pull image Docker	<pre># Login docker login artifactory.mon-domaine.com \ -u \$user -p \$token # Tag + Push docker tag mon-app:1.0 \ artifactory.mon-domaine.com/docker-local-prod/mon-app:1.0 docker push \ artifactory.mon-domaine.com/docker-local-prod/mon-app:1.0 # Depuis un pipeline Azure DevOps task: Docker@2 inputs: containerRegistry: 'JFrog-ServiceConnection' repository: 'docker-local-prod/mon-app' command: 'buildAndPush' tags: '\$(Build.BuildId)'</pre>	Configurer Service Connection Azure DevOps Utiliser artifactory-build-info pour traçabilité Promotion : dev → staging → prod via dépôts

Opération	Commande / Script PowerShell	Description & Bonnes Pratiques
Secret K8s pour JFrog	<pre># Créer imagePullSecret kubectl create secret docker-registry jfrog-registry \ --docker-server=artifactory.mon-domaine.com \ --docker-username=\$user \ --docker-password=\$token \ --namespace=mon-app # Référencer dans le Deployment spec: imagePullSecrets: - name: jfrog-registry containers: - name: app image: artifactory.mon-domaine.com/docker-local-prod/mon-app:1.0</pre>	Alternative : attacher JFrog à l'identité managée AKS ou utiliser External Secrets Operator Roter les tokens régulièrement (TTL 30-90 jours)
Déployer chart Helm	<pre># Ajouter repo Helm JFrog helm repo add mon-jfrog \ https://artifactory.mon-domaine.com/artifactory/helm-virtual \ --username \$user --password \$token helm repo update # Publier un chart \$chartPath = './charts/mon-app-1.0.0.tgz' curl -u "\$user:\$token" \ -T \$chartPath \ "\$env:JFROG_URL/artifactory/helm-local/mon-app-1.0.0.tgz" # Déployer depuis JFrog helm install mon-app mon-jfrog/mon-app \ --version 1.0.0 \ --namespace mon-app</pre>	Helm chart = artefact versionné dans JFrog Index automatique du repo Helm CI/CD : packager + publier à chaque build
JFrog Xray (Sécurité)	<pre># Configurer une politique de sécurité (API) \$policy = @{ name = 'block-critical' type = 'security' rules = @(@{ name = 'critical-cve' criteria = @{ min_severity = 'Critical' } actions = @{ block_download = @{ active = \$true } fail_build = \$true } }) } ConvertTo-Json -Depth 10 Invoke-RestMethod \ -Uri "\$env:JFROG_URL/xray/api/v2/policies" \ -Method POST -Headers \$headers -Body \$policy</pre>	Xray : scan CVE des images + artefacts Intégré dans le pipeline CI/CD Bloquer le téléchargement si CVE critique Rapports de conformité (licences)
Nettoyage & Rétention	<pre># Supprimer artefacts anciens via AQL \$aql = 'items.find({"repo": {"\$eq": "docker-local-prod"}, "created": {"\$before": "3mo"}}).include("name","repo","path")' \$result = Invoke-RestMethod \ -Uri "\$env:JFROG_URL/artifactory/api/search/aql" \ -Method POST -Headers \$headers -Body \$aql # Activer la politique de rétention automatique # Admin → Artifacts Cleanup → New Policy # → Critères : âge > 90 jours, tags non-latest</pre>	AQL = Artifactory Query Language Politique de rétention : Admin UI ou API Garder les images taggées (semver) Purger les builds :latest non-promus

⑤ CI/CD Pipeline Azure DevOps → JFrog → AKS/OpenShift

Construction d'un pipeline CI/CD de bout en bout avec Azure DevOps : build et push de l'image Docker vers JFrog, scan de sécurité avec Xray qui bloque le pipeline si CVE critique détectée, puis déploiement automatisé sur AKS (via KubernetesManifest) et sur OpenShift (via oc set image) avec attente de la bonne santé du rollout.

Étape	Configuration YAML Azure DevOps	Description
Build & Push image	<pre>trigger: branches: include: [main, release/*] pool: vmImage: ubuntu-latest stages: - stage: Build jobs: - job: BuildImage steps: - task: Docker@2 displayName: 'Build & Push to JFrog' inputs: containerRegistry: 'JFrog-Service-Connection' repository: 'docker-local-prod/mon-app' command: 'buildAndPush' dockerfile: '\$(Build.SourcesDirectory)/Dockerfile' tags: \$(Build.BuildId) latest</pre>	Service Connection configurée dans Azure DevOps Project Settings → Service Connections → Docker Registry URL : https://artifactory.mon-domaine.com
Scan sécurité Xray	<pre>- script: # Installer JFrog CLI curl -fL https://install-cli.jfrog.io sh jfrog config add server1 \ --artifactory-url=\$JFROG_URL \ --access-token=\$JFROG_TOKEN # Build info + scan jfrog rt build-collect-env jfrog rt docker-push \ artifactory.mon-domaine.com/docker-local-prod/mon-app:\$(Build.BuildId) \ docker-local-prod --build-name=\$(Build.DefinitionName) \ --build-number=\$(Build.BuildId) jfrog rt build-publish jfrog xr build-scan \ \$(Build.DefinitionName) \$(Build.BuildId) \ --fail=true env: JFROG_TOKEN: \$(JFROG_TOKEN)</pre>	JFrog CLI = outil officiel pour CI/CD Build info = traçabilité complète des artefacts Xray scan bloque si CVE critique détectée Variable secrète stockée dans Azure DevOps Library
Déployer sur AKS	<pre>- stage: DeployAKS dependsOn: Build condition: succeeded() jobs: - deployment: Deploy environment: 'aks-production' strategy: runOnce: deploy: steps: - task: KubernetesManifest@0 displayName: 'Deploy to AKS' inputs: action: deploy kubernetesServiceConnection: 'AKS-Prod-Connection' namespace: mon-app manifests: \$(Pipeline.Workspace)/manifests/deployment.yaml \$(Pipeline.Workspace)/manifests/service.yaml containers: artifactory.mon-domaine.com/docker-local-prod/mon-app:\$(Build.BuildId)</pre>	Environment = gate d'approbation manuelle kubernetesServiceConnection → AKS Injecte le bon tag d'image automatiquement Rollback automatique si health check échoue
Déployer sur OpenShift	<pre>- stage: DeployARO jobs: - job: DeployOCP steps: - script: # Login OpenShift oc login \$OCP_URL \ --token=\$OCP_TOKEN \ --insecure-skip-tls-verify=false # Mettre à jour l'image oc set image deployment/mon-app \ app=artifactory.mon-domaine.com/docker-local-prod/mon-app:\$(Build.BuildId) \ -n mon-projet # Attendre le rollout oc rollout status deployment/mon-app \ -n mon-projet --timeout=300s env: OCP_TOKEN: \$(OCP_TOKEN)</pre>	Token OCP stocké dans Azure DevOps Library oc set image = mise à jour rolling sans downtime oc rollout status bloque jusqu'à succès ou timeout

■ Bonnes pratiques CI/CD : utiliser des tags sémantiques (v1.2.3) au lieu de latest en production · stocker les secrets dans Azure Key Vault (non dans les variables de pipeline) · activer la protection d'environnement avec approbateurs · scanner les images avant déploiement.

⑥ ArgoCD — GitOps sur AKS & OpenShift

Introduction au paradigme GitOps avec ArgoCD : Git comme source de vérité unique, synchronisation continue entre le dépôt de configuration et l'état du cluster. Cette section couvre les concepts clés, l'installation, la création d'applications, les opérations quotidiennes et l'intégration complète avec le pipeline CI/CD Azure DevOps.

■ ArgoCD et cert-manager sont complémentaires : ArgoCD déploie et protège les objets cert-manager (ClusterIssuer, Certificate) depuis Git via self-heal, tandis que cert-manager réalise l'émission cryptographique des certificats TLS. L'intégration complète avec les Sync Waves et la structure de repo GitOps est détaillée dans le Volume 3 — Section ⑥.

Concepts Fondamentaux GitOps / ArgoCD

Les 6 notions à maîtriser avant d'utiliser ArgoCD : le paradigme GitOps, l'objet Application (lien Git → cluster), la Sync Policy (manuelle vs automatique avec self-heal), le pattern App of Apps pour gérer tout un cluster, les ApplicationSets pour générer des déploiements multi-environnements, et les Sync Waves & Hooks pour séquencer les ressources.

Concept	Définition	Exemple concret
GitOps	Paradigme où Git est la source de vérité unique. Toute modification passe par un commit/PR. L'opérateur GitOps détecte et applique les diffs automatiquement.	Développeur fait un PR → CI valide → merge → ArgoCD détecte le changement → sync automatique sur le cluster
Application ArgoCD	Objet K8s (CRD) qui lie un dépôt Git (source) à un cluster/namespace (destination). ArgoCD surveille en permanence les deux.	<code>kind: Application</code> source: repoURL: https://github.com/org/repo path: k8s/prod destination: server: https://kubernetes.default.svc namespace: mon-app
Sync Policy	Définit si la synchronisation est manuelle (opérateur valide) ou automatique (ArgoCD applique tout commit). Self-heal : corrige les dérives manuelles.	<code>syncPolicy: automated: prune: true</code> # supprime ressources orphelines <code>selfHeal: true</code> # corrige les dérives manuelles <code>syncOptions: - CreateNamespace=true</code>
App of Apps	Pattern où une Application ArgoCD (root) déploie d'autres Applications. Permet de bootstrapper tout un cluster depuis un seul repo Git.	# repo/ # ■■■■ root-app.yaml ← Application racine # ■■■■ apps/ # ■■■■ mon-app.yaml ← Application 1 # ■■■■ monitoring.yaml ← Application 2 # ■■■■ ingress.yaml ← Application 3
ApplicationSet	Génère automatiquement des Applications depuis des templates. Utile pour multi-cluster, multi-environnement (dev/staging/prod) depuis un seul manifest.	<code>kind: ApplicationSet</code> spec: generators: - list: elements: - cluster: dev url: https://dev-aks - cluster: prod url: https://prod-aks template: spec: source: path: 'k8s/{{cluster}}'
Sync Waves & Hooks	Contrôler l'ordre de déploiement des ressources. Les Hooks (PreSync, Sync, PostSync) permettent d'exécuter des jobs (ex: migration BDD) à des étapes clés.	# Ordre via annotation wave metadata: annotations: argocd.argoproj.io/sync-wave: '1' # avant wave 2,3... # Hook PreSync = migration BDD avant déploiement metadata: annotations: argocd.argoproj.io/hook: PreSync argocd.argoproj.io/hook-delete-policy: HookSucceeded

Installation & Configuration ArgoCD sur AKS

Déploiement d'ArgoCD via Helm sur AKS avec Ingress NGINX et certificat TLS automatique (cert-manager). Configuration du SSO Azure Active Directory pour que l'équipe se connecte avec son compte entreprise, connexion des dépôts Git privés (HTTPS/SSH), et gestion multi-cluster en mode hub-and-spoke.

Étape	Commande / YAML	Notes
Installer ArgoCD (Helm recommandé)	<pre>helm repo add argo https://argoproj.github.io/argo-helm helm repo update helm install argocd argo/argo-cd \ --namespace argocd \ --create-namespace \ --version 7.x.x \ --set configs.params.server.insecure=false \ --set server.ingress.enabled=true \ --set server.ingress.ingressClassName=nginx \ --set server.ingress.annotations."cert-manager.io/cluster-issuer"=letsencrypt-prod \ --set "server.ingress.hosts[0]"=argocd.mon-domaine.com \ --set "server.ingress.tls[0].secretName"=argocd-tls \ --set "server.ingress.tls[0].hosts[0]"=argocd.mon-domaine.com # Vérifier les pods kubectl get pods -n argocd</pre>	Ingress NGINX + cert-manager TLS intégrés Expose l'UI ArgoCD sur https://argocd.mon-domaine.com Disable insecure pour forcer HTTPS Version : voir https://github.com/argoproj/argo-helm
Récupérer le mot de passe admin initial	<pre># Récupérer le secret admin \$pw = kubectl get secret argocd-initial-admin-secret \ -n argocd \ -o jsonpath='{.data.password}' \ ForEach-Object { [Text.Encoding]::UTF8.GetString([Convert]::FromBase64String(\$_)) } Write-Host "ArgoCD admin password: \$pw" # Login via CLI argocd login argocd.mon-domaine.com \ --username admin \ --password \$pw \ --grpc-web # Changer le mot de passe argocd account update-password</pre>	Supprimer le secret après avoir noté le mot de passe : kubectl delete secret argocd-initial-admin-secret -n argocd Créer des comptes utilisateurs ou intégrer AAD (SSO)
Intégration Azure AD (SSO)	<pre># Dans values.yaml Helm (OIDC AAD) configs: cm: url: https://argocd.mon-domaine.com oidc: config: name: Azure AD issuer: https://login.microsoftonline.com/<tenant-id>/v2.0 clientID: <app-client-id> clientSecret: \$oidc.azure.clientSecret requestedScopes: [openid, profile, email] requestedIDTokenClaims: groups: essential: true rbac: policy.csv: g, <AAD-Group-DevOps-ObjectId>, role:admin g, <AAD-Group-Dev-ObjectId>, role:readonly policy.default: role:readonly # Secret OIDC kubectl create secret generic argocd-oidc-azure \ --from-literal=clientSecret='<app-secret>' \ -n argocd</pre>	SSO via Azure AD = login avec compte entreprise Groupes AAD mappés sur rôles ArgoCD role:admin, role:readonly, ou rôles custom Plus besoin de gérer des comptes ArgoCD locaux

Étape	Commande / YAML	Notes
Connecter un dépôt Git (privé)	<pre># Via CLI (HTTPS + token) argocd repo add https://github.com/org/k8s-config \ --username git \ --password \$GITHUB_PAT # Via CLI (SSH) argocd repo add git@github.com:org/k8s-config.git \ --ssh-private-key-path ~/.ssh/id_rsa # Via kubectl (Secret K8s) kubectl create secret generic repo-k8s-config \ --from-literal=type=git \ --from-literal=url=https://github.com/org/k8s-config \ --from-literal=password=\$GITHUB_PAT \ --from-literal=username=git \ -n argocd kubectl label secret repo-k8s-config \ argocd.argoproj.io/secret-type=repository -n argocd</pre>	PAT GitHub/Azure DevOps/GitLab selon la plateforme SSH = clé déployée sur le dépôt (deploy key) Préférer SSH en production Azure DevOps : PAT avec scope Code Read
Connecter un cluster externe	<pre># Lister les contextes kubeconfig disponibles argocd cluster add aks-staging \ --name aks-staging \ --kubeconfig ~/.kube/config # Vérifier les clusters connectés argocd cluster list # Ou via kubectl Secret kubectl apply -f - <<EOF apiVersion: v1 kind: Secret metadata: name: aks-staging-cluster namespace: argocd labels: argocd.argoproj.io/secret-type: cluster type: Opaque stringData: name: aks-staging server: https://aks-staging.hcp.eastus.azmk8s.io config: '{"bearerToken":"<token>","tlsClientConfig":{"insecure":false}}' EOF</pre>	ArgoCD hub-and-spoke : 1 cluster ArgoCD central gère N clusters cibles Service Account créé dans chaque cluster cible Token bearerToken = SA token du cluster distant

Créer & Gérer des Applications ArgoCD

Cinq scénarios concrets de déploiement : depuis des manifests YAML bruts ou Kustomize, depuis un chart Helm hébergé sur JFrog, génération automatique d'applications pour plusieurs environnements avec ApplicationSet, bootstrapping complet d'un cluster via App of Apps, et ordonnancement des ressources avec Sync Waves (namespace → secrets → migration BDD → application).

Scénario	Manifest / Commande	Détails
Déployer depuis manifests YAML bruts	<pre>apiVersion: argoproj.io/v1alpha1 kind: Application metadata: name: mon-app-prod namespace: argocd spec: project: default source: repoURL: https://github.com/org/k8s-config targetRevision: main path: apps/mon-app/overlays/prod destination: server: https://kubernetes.default.svc namespace: mon-app syncPolicy: automated: prune: true selfHeal: true syncOptions: - CreateNamespace=true - PrunePropagationPolicy=foreground retry: limit: 5 backoff: duration: 5s factor: 2 maxDuration: 3m</pre>	targetRevision: main → suit la branche Ou tag : v1.2.3 → déploiement immuable (recommandé prod) path : dossier Kustomize ou YAML dans le repo retry : réessaie 5 fois avec backoff exponentiel
Déployer depuis un chart Helm	<pre>apiVersion: argoproj.io/v1alpha1 kind: Application metadata: name: mon-app-helm namespace: argocd spec: project: production source: repoURL: https://artifactory.mon-domaine.com/artifactory/helm-virtual chart: mon-app targetRevision: 1.2.0 helm: releaseName: mon-app valueFiles: - values-prod.yaml parameters: - name: image.tag value: v1.2.0 - name: replicaCount value: '3' destination: server: https://kubernetes.default.svc namespace: mon-app syncPolicy: automated: selfHeal: true</pre>	Supporte Helm depuis JFrog Artifactory chart + targetRevision → version fixe du chart valueFiles : fichiers values dans le repo Git parameters : override inline (utile en CI/CD)
ApplicationSet multi-environnements	<pre>apiVersion: argoproj.io/v1alpha1 kind: ApplicationSet metadata: name: mon-app-all-envs namespace: argocd spec: goTemplate: true generators: - list: elements: - env: dev cluster: https://dev-aks.hcp.eastus.azure.com namespace: mon-app-dev replicas: '1' - env: staging cluster: https://staging-aks.hcp.eastus.azure.com namespace: mon-app-staging replicas: '2' - env: prod cluster: https://prod-aks.hcp.eastus.azure.com namespace: mon-app replicas: '3' template: metadata: name: 'mon-app-{{.env}}' spec: project: default source: repoURL: https://github.com/org/k8s-config path: 'apps/mon-app/overlays/{{.env}}' targetRevision: main destination: server: '{{.cluster}}' namespace: '{{.namespace}}' syncPolicy: automated: selfHeal: true</pre>	1 ApplicationSet → 3 Applications générées automatiquement (dev, staging, prod) Chaque environnement peut avoir ses propres paramètres Ajouter un environnement = ajouter un élément à la liste

Scénario	Manifest / Commande	Détails
App of Apps (bootstrapping cluster)	<pre># repo/ # ■■■ bootstrap/ # ■■■ root-app.yaml ← appliquer UNE SEULE FOIS # ■■■ apps/ # ■■■ argocd-app.yaml # ■■■ ingress-nginx-app.yaml # ■■■ cert-manager-app.yaml # ■■■ external-secrets-app.yaml # ■■■ mon-app.yaml # root-app.yaml apiVersion: argoproj.io/v1alpha1 kind: Application metadata: name: root-app namespace: argocd spec: source: repoURL: https://github.com/org/cluster-config path: bootstrap/apps targetRevision: main destination: server: https://kubernetes.default.svc namespace: argocd syncPolicy: automated: selfHeal: true prune: true # Bootstrapper un nouveau cluster : # kubectl apply -f bootstrap/root-app.yaml # → ArgoCD déploie tout le reste automatiquement</pre>	Pattern le plus puissant pour l'IaC K8s Un seul kubectl apply suffit pour tout un cluster Tout est versionné dans Git (infra as code) Idéal pour disaster recovery ou nouveaux environnements
Déploiement avec Sync Waves & Hooks	<pre># Wave 0 : Namespace + RBAC apiVersion: v1 kind: Namespace metadata: name: mon-app annotations: argocd.argoproj.io/sync-wave: '0' --- # Wave 1 : Secrets + ConfigMaps apiVersion: v1 kind: ConfigMap metadata: name: app-config annotations: argocd.argoproj.io/sync-wave: '1' --- # Wave 2 : Job migration BDD (Hook PreSync) apiVersion: batch/v1 kind: Job metadata: name: db-migrate annotations: argocd.argoproj.io/hook: PreSync argocd.argoproj.io/hook-delete-policy: HookSucceeded argocd.argoproj.io/sync-wave: '2' --- # Wave 3 : Deployment application apiVersion: apps/v1 kind: Deployment metadata: name: mon-app annotations: argocd.argoproj.io/sync-wave: '3'</pre>	Sync Waves : ordre garanti de déploiement 0 < 1 < 2 < 3 (attendre que wave N soit Healthy) PreSync Hook : s'exécute AVANT le sync principal HookSucceeded : supprime le Job si succès Idéal pour migrations BDD avant mise à jour app

CLI ArgoCD & Opérations Quotidiennes

Référence des commandes `argocd` et `kubectl` pour le travail quotidien : consulter le statut et la santé des applications, déclencher une synchronisation manuelle ou sélective, visualiser les diffs et l'historique, effectuer un rollback, isoler les équipes avec les `AppProjects`, configurer les alertes `Teams/Slack`, et utiliser `ArgoCD` via l'`OpenShift GitOps Operator`.

Catégorie	Commande <code>argocd</code> / <code>kubectl</code>	Description
Statut & Santé	<pre># Lister toutes les applications argocd app list # Statut détaillé argocd app get mon-app-prod # Via kubectl kubectl get applications -n argocd -o custom-columns='NAME:.metadata.name,SYNC:.status.sync,status:health.status'</pre>	SYNC : Synced / OutOfSync / Unknown HEALTH : Healthy / Degraded / Progressing / Missing OutOfSync = Git ≠ cluster (diff détecté)
Synchroniser	<pre># Sync manuel argocd app sync mon-app-prod # Sync avec pruning (supprime ressources orphelines) argocd app sync mon-app-prod --prune # Sync d'une ressource spécifique uniquement argocd app sync mon-app-prod \ --resource apps:Deployment:mon-app # Forcer le refresh (re-fetch du repo Git) argocd app get mon-app-prod --refresh</pre>	--prune : supprime les ressources supprimées dans Git --resource : sync sélectif (ex: juste un Deployment) --refresh : force la re-lecture du repo Git En mode automated : le sync se fait automatiquement
Diff & Historique	<pre># Voir le diff Git vs cluster argocd app diff mon-app-prod # Historique des déploiements argocd app history mon-app-prod # Rollback à une révision précédente argocd app rollback mon-app-prod <revision-id> # Voir les ressources gérées argocd app resources mon-app-prod</pre>	diff = ce qui sera appliqué si on sync history = toutes les synchronisations passées rollback = revenir à un état Git précédent Ne pas confondre avec <code>kubectl rollout undo</code>
Gestion des Projets (AppProject)	<pre># Créer un projet avec restrictions apiVersion: argoproj.io/v1alpha1 kind: AppProject metadata: name: production namespace: argocd spec: description: Applications de production sourceRepos: - https://github.com/org/k8s-config - https://artifactory.mon-domaine.com/artifactory/helm-virtual destinations: - namespace: 'mon-app*' server: https://prod-aks.hcp.eastus.azmk8s.io clusterResourceWhitelist: - group: '' kind: Namespace namespaceResourceBlacklist: - group: '' kind: ResourceQuota # ne peut pas modifier les quotas</pre>	AppProject = périmètre de sécurité dans ArgoCD Restrictions : repos Git autorisés, clusters cibles, namespaces, types de ressources autorisés/interdits Idéal pour isolation équipes (dev ne touche pas prod)

Catégorie	Commande argocd / kubectl	Description
Notifications ArgoCD (Teams / Slack)	<pre># Installer le controller de notifications helm upgrade argocd argo/argo-cd \ --set notifications.enabled=true \ -n argocd # Secret Teams/Slack webhook kubectl create secret generic argocd-notifications-secret \ --from-literal=teams-webhook-url='https://outlook.office.com/webhook/...' \ -n argocd # ConfigMap notifications apiVersion: v1 kind: ConfigMap metadata: name: argocd-notifications-cm namespace: argocd data: service.teams: recipientUrls: general: \$teams-webhook-url trigger.on-sync-failed: - when: app.status.operationState.phase in ['Error','Failed'] send: [app-sync-failed] template.app-sync-failed: message: Application {{.app.metadata.name}} sync FAILED! Revision: {{.app.status.sync.revision}}</pre>	Triggers : on-sync-succeeded, on-sync-failed, on-health-degraded, on-deployed Canaux : Slack, Teams, Email, PagerDuty, OpsGenie Annotation sur l'Application pour activer : notifications.argoproj.io/subscribe.on-sync-failed.teams: general
ArgoCD sur OpenShift (ARO)	<pre># Installer via OperatorHub sur OCP # OpenShift GitOps Operator (basé sur ArgoCD) oc get csv -n openshift-gitops # L'instance ArgoCD est créée automatiquement oc get argocd -n openshift-gitops # Accéder à la console oc get route openshift-gitops-server -n openshift-gitops # Récupérer le mot de passe admin oc extract secret/openshift-gitops-cluster \ -n openshift-gitops --to=- # Donner les droits cluster-admin à ArgoCD # (pour déployer sur tout le cluster) oc adm policy add-cluster-role-to-user cluster-admin \ -z openshift-gitops-argocd-application-controller \ -n openshift-gitops</pre>	OpenShift GitOps = distribution Red Hat d'ArgoCD Installé via OperatorHub (1 clic) Intégration SSO AAD via OpenShift OAuth Routes OCP créées automatiquement pour l'UI SCC gérés automatiquement

Intégration CI/CD Complète : Azure DevOps + JFrog + ArgoCD

Pipeline de référence en 4 étapes pour séparer CI et CD : Azure DevOps construit et scanne l'image (CI), met à jour le dépôt de configuration K8s avec le nouveau tag (déclencheur GitOps), ArgoCD détecte le changement et synchronise automatiquement le cluster (CD), puis ArgoCD Rollouts assure un déploiement progressif Canary avec analyse automatique des métriques avant validation complète.

Étape	Rôle	Script / YAML
1. CI — Build & Push (Azure DevOps)	Pipeline CI : build image, scan Xray, push vers JFrog Artifactory avec le tag du build	<pre># azure-pipelines.yml - stage: CI jobs: - job: BuildAndPush steps: - task: Docker@2 inputs: command: buildAndPush containerRegistry: JFrog-Connection repository: docker-local-prod/mon-app tags: \$(Build.BuildId) - script: jfrog xr build-scan \$(Build.DefinitionName) \$(Build.BuildId) displayName: 'Xray Security Scan'</pre>

Étape	Rôle	Script / YAML
2. CI — Mise à jour du repo de config Git	Après validation, le pipeline CI met à jour le tag d'image dans le dépôt de configuration K8s (kustomize ou values.yaml Helm). C'est le déclencheur GitOps.	<pre>- script: # Cloner le repo de config git clone https://\$(GIT_PAT)@github.com/org/k8s-config.git cd k8s-config # Mettre à jour le tag dans kustomize cd apps/mon-app/overlays/prod kustomize edit set image \ mon-app=artifactory.mon-domaine.com/docker-local-prod/mon-app:\$(Build.BuildId) # Commit & Push git config user.email 'ci@org.com' git config user.name 'Azure DevOps CI' git add . git commit -m 'chore: update mon-app image to \$(Build.BuildId) [skip ci]' git push displayName: 'Update K8s config repo'</pre>
3. CD — ArgoCD détecte le changement	ArgoCD surveille le repo de config en continu. Dès le commit, il détecte l'OutOfSync et applique automatiquement le nouveau manifeste.	<pre># ArgoCD Application en mode automated syncPolicy: automated: prune: true selfHeal: true # Ou déclencher manuellement depuis le pipeline CI - script: argocd login argocd.mon-domaine.com \ --auth-token \$(ARGOCD_TOKEN) --grpc-web argocd app sync mon-app-prod --timeout 300 argocd app wait mon-app-prod \ --health --timeout 300 displayName: 'ArgoCD Sync & Wait'</pre>
4. CD — Progressive Delivery (Rollout)	ArgoCD Rollouts (extension) : Canary ou Blue-Green avec analyse automatique via métriques Prometheus/Azure Monitor.	<pre>apiVersion: argoproj.io/v1alpha1 kind: Rollout metadata: name: mon-app spec: strategy: canary: steps: - setWeight: 10 # 10% du trafic sur nouvelle version - pause: {duration: 5m} - analysis: templates: - templateName: success-rate - setWeight: 50 - pause: {duration: 5m} - setWeight: 100 template: spec: containers: - name: app image: artifactory.mon-domaine.com/docker-local-prod/mon-app:v1.2</pre>

■ Architecture GitOps recommandée : 2 dépôts Git séparés (app-code et k8s-config) · CI = Azure DevOps construit l'image et met à jour k8s-config · CD = ArgoCD surveille k8s-config et synchronise le cluster · Aucun accès kubectl direct en production (tout passe par Git) · ArgoCD Rollouts pour les déploiements progressifs Canary/Blue-Green · Notifications Teams/Slack sur chaque sync et dégradation de santé.